

多模态“卷王”阶跃星辰：如何利用 JuiceFS 打造高效经济的大模型存储平台

在业界有“多模态卷王”之称的阶跃星辰，自研的 22 款基础模型中有 16 款为多模态模型，覆盖文字、语音、图像、视频、音乐与推理等多个方向。为支撑多模态模型的研发与落地，团队在基础设施层需覆盖从数据生成、清洗，到模型预训练、后训练、推理部署的完整链路，对存储系统在数据吞吐、访问延迟与读写效率等方面提出了极高要求。

随着业务规模的快速扩展，早期采用的商业存储系统逐渐暴露出扩展性有限、稳定性不足、运维复杂等问题，难以满足多模态大模型持续增长的存储需求。为此，团队开始探索更加轻量、灵活、可控且具备成本优势的替代方案。JuiceFS 以其良好的易用性迅速接入自研的对象存储，并逐步演化为统一的文件系统接入层，对接高性能与大容量存储，实现了冷热数据的自动迁移与透明流动。

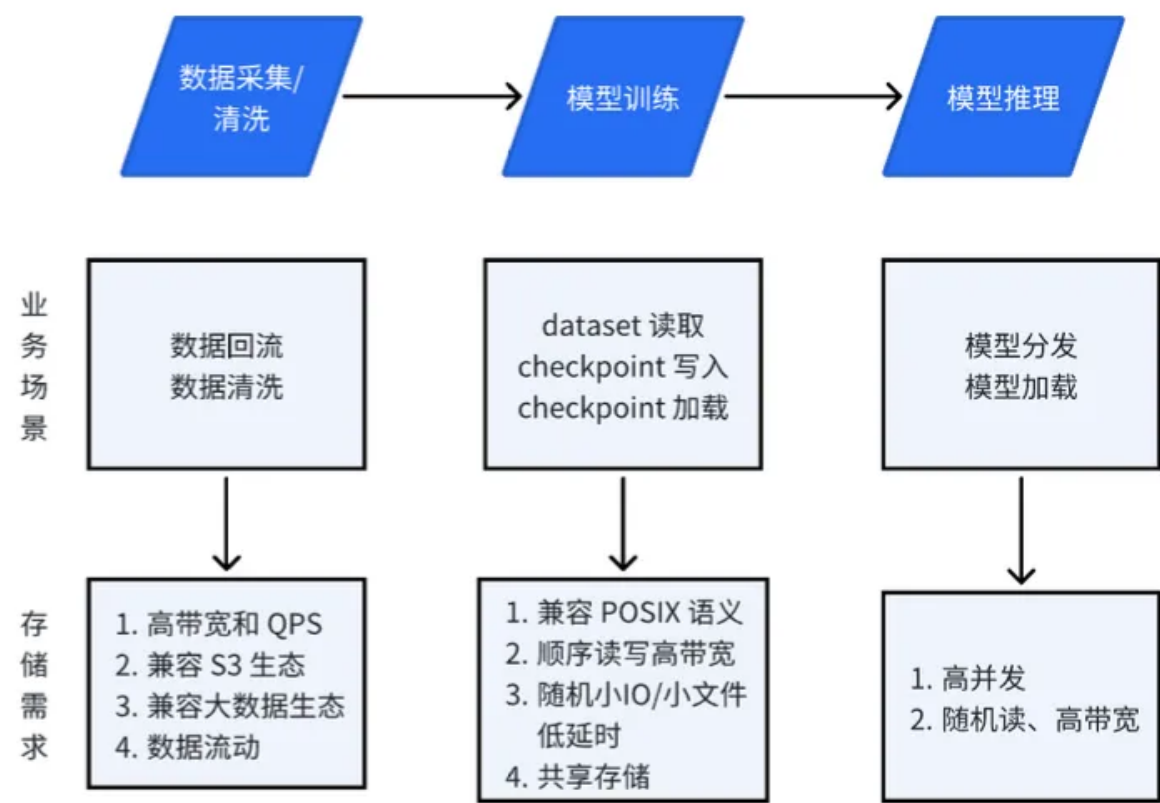
目前，阶跃星辰在生产环境中部署 JuiceFS 社区版与企业版，覆盖模型训练、推理部署等核心场景。值得一提的是，本文作者缪昌新也是 JuiceFS 社区的长期贡献者，自 2021 年项目开源以来持续参与开发，并在最近发布的 1.3 版本中贡献了多项 TiKV 性能优化改进。本文将详细介绍他所在团队基于社区版进行的多项优化实践，分享他们在企业级多模态大模型落地中的一线经验。

大模型对存储的需求

数据采集与清洗

在数据采集与清洗环节，公司设有专门的数据团队，负责处理回流数据和清洗任务。这类业务对存储系统提出了较高的性能要求，尤其关注高带宽与高 QPS。实际运行中，数据采集过程常常需要 TB 级别的带宽，系统带宽几乎被打满，访问并发量也持续处于高位。

此外，大数据处理也是核心场景之一。无论是 Spark 等计算框架、Ray 任务调度，还是各类数据分析流程，都高度依赖底层存储系统，对其吞吐能力、稳定性与扩展性提出了更高要求。



大模型研发各场景的存储需求

模型训练

在模型训练过程中，存储系统的核心需求体现在数据集的高效读取。读取延迟通常需控制在微秒到毫秒级别，否则将显著拖慢训练进度。例如，一个原本需要两个月完成的预训练任务，若存储读取速度下降 10%，可能会导致训练周期延长至少一周。

不同的数据组织方式也会带来截然不同的 IO 访问模式：打包后的数据集面临高并发的随机读取挑战，而未打包的数据集则涉及大量小文件的处理，对文件系统的元数据性能提出更高要求。

另一个关键挑战是 Checkpoint 的读写性能。训练过程中的 Checkpoint 操作以顺序读写为主，且大模型的 Checkpoint 规模可能达到 TB 级别。这不仅要求存储系统具备高速写入能力，还需在故障恢复时实现快速回读，因此对读写带宽提出了极高要求。

模型推理

模型上线阶段对存储性能同样提出了较高要求。尽管模型在量化后体积有所缩小，但 TB 级别的模型量化后依然会有数百 GB，例如 Deepseek 的量化模型大约 600 GB。主流云厂商的 GPU 资源常面临潮汐式负载波动，当流量突增时需进行动态扩容，而模型加载往往成为扩容过程中的主要瓶颈。因此，存储系统必须支持高并发访问和大带宽读取，以保障模型能够在短时间内加载完成。

总的来说，大模型研发在各个环节对存储需求很高，不仅要求存储提供高带宽，还需要满足低延迟的处理能力。大模型对存储的核心诉求可总结为以下几点：

- **元信息**：需支持百亿到千亿级文件数量，满足大模型在数据处理和训练中对大规模小文件管理的需求。
- **大容量**：单个文件系统的容量需覆盖 PB 至数十 PB 级别，以承载长期积累的训练数据和模型存档。
- **低延时**：要求对随机小 IO 和小文件访问具备 us 到 ms 级别的响应能力，保障数据加载过程中的低延时。
- **高带宽**：在训练过程中，如 Checkpoint 的顺序读写通常达到百 GB 级别，存储系统需提供相应的持续带宽能力。
- **POSIX**：需支持标准 POSIX 语义，便于算法工程师直接以本地文件方式访问数据，降低他们的工程负担。
- **低成本**：需支持高性能低容量与低性能高容量介质间的数据流转，实现冷热数据分层，降低整体存储成本。

Why JuiceFS

我们在大模型研发全流程中引入 JuiceFS，最初主要是基于以下几个方面的考量。

灵活适配

JuiceFS 可以无缝对接多云环境，并支持我们自建机房的基础设施。在大模型场景中，数据存储与计算紧密相关，存储始终跟随计算节点分布，哪里有计算资源，我们就把数据搬到哪里。因此，灵活适配性是我们非常看重的一个特点。

极简开发

另一个优势是极简开发。存储系统通常需要一定的定制化，JuiceFS 的二次开发门槛非常低。其代码量大约为 8 万行，相较于传统的存储系统（如使用 C++ 的方案），开发复杂度大大降低。这使得我们的二次开发工作更加高效和简便。

全栈兼容

我们也非常重视 POSIX 语义兼容性，JuiceFS 在这方面的表现非常优秀，能够确保我们在多种使用姿势下的稳定性和兼容性。此外，它能无缝对接 HDFS 和 S3 Gateway，极大简化了我们内部数据流动的操作流程，使得数据交换和管理更加高效。

我们同时使用了 JuiceFS 的企业版和社区版，下面将分别进行介绍。

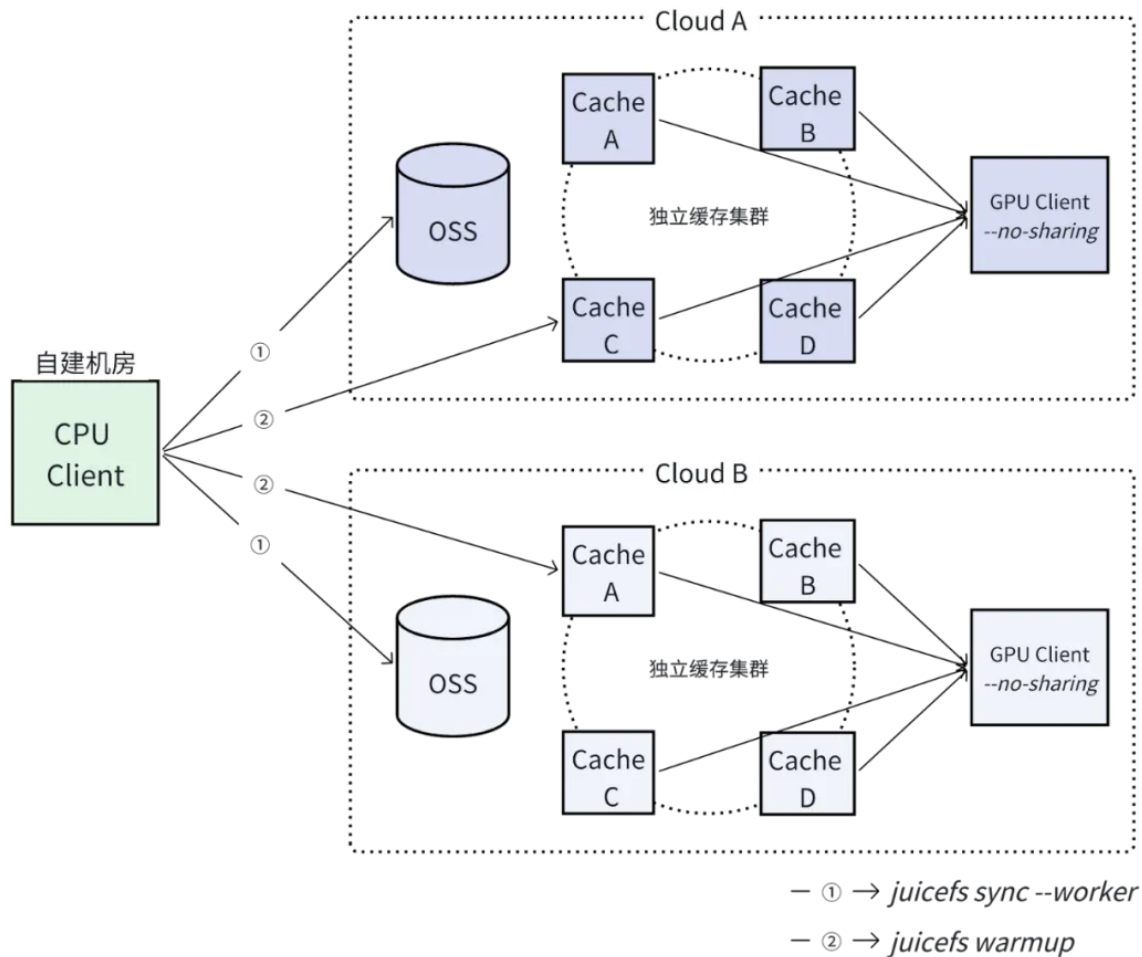
JuiceFS 企业版的使用与优化

多云推理部署中的模型分发与缓存优化

在大模型推理部署阶段，我们基于 JuiceFS 企业版构建了一套适配多云环境的模型同步与缓存体系。其核心目标是实现模型的高效跨云分发与快速加载预热，以应对多云资源调度下的动态推理需求。

具体流程如下：用户发布模型时，系统会根据各云厂商的资源使用情况，将模型定向同步至目标云环境。我们采用 JuiceFS 提供的分布式同步工具 Juicesync，可将百 GB 级模型快速、可靠地分发到多个机房的 JuiceFS 中，显著降低了跨云数据传输的成本与复杂度。

模型同步完成后，系统会执行 Warmup 操作，通过 JuiceFS 将模型文件预加载到推理路径中的缓存节点。预热机制有效缓解了由于模型体积庞大而导致的冷启动延迟问题，显著提升了扩缩容场景下的响应速度和推理稳定性。



多云环境下 JuiceFS 模型推理部署架构

需要注意的是，部分云厂商的 GPU 节点不配备本地 NVMe 硬盘，即使有盘的 GPU 节点受资源波动或者硬件故障的影响，运行中可能频繁上下线，不适合承担缓存任务。为确保缓存的持久性与稳定性，我们将缓存服务部署在少量稳定的非 GPU 通用计算节点上，并借助 JuiceFS 的灵活挂载机制实现与计算节点解耦的独立管理。该方案已在多个云平台上稳定运行，成为支撑多云推理架构的关键存储基础之一。

safetensors 随机读性能优化

在推理场景中，我们遇到了一个典型的问题 —— 读放大。模型上线时通常需要转换为 safetensors 格式，该格式在加载过程中会触发大量的 mmap 随机读。这类负载并非 JuiceFS 的优势场景，尽管缓存命中率已达到 100%，但在实际使用中，用户仍明显感受到读带宽受限，系统资源未被充分利用。

从下图监控数据可见，用户侧接收到的读带宽仅在 3-5 GB/s 区间波动，而后端 Remote Cache 的吞吐能力稳定维持在 25 GB/s 左右，且带宽曲线平稳，表明后端资源已逼近性能上限。二者之间的差距主要源于 JuiceFS 的 readahead 策略 —— **系统预读取的数据远超用户实际所需，导致读放大严重，带宽浪费严重**。这意味着为了满足当前的读取需求，系统不得不付出数倍的额外成本。



优化前读性能情况

下图展示了优化前后的系统读放大变化。优化前，读放大率一度高达 1042%，意味着系统为了读取实际所需数据付出了 10 倍以上的 IO 成本。优化之后，读放大显著下降，长期稳定在 300% 左右，系统读取效率和带宽利用率均得到显著提升。



优化后读性能情况

以下是我们在与 JuiceFS 团队深入沟通后制定并实施的具体优化措施：

首先，我们启用了二级缓存机制，其中第一级缓存使用 GPU 机器的内存，能够将读放大控制在本地内存中，避免对远端带宽造成压力。为此，我们通过设置参数 `cache-dir=memory` 将缓存直接放入内存，并将 `cache-size` 配置为 10240，以充分利用 GPU 机器较大的内存资源。

其次，我们关闭了 JuiceFS 默认的 `readahead` 机制，通过将 `max-readahead` 设置为 1，避免系统预取无效数据，从而进一步减少随机读带来的开销。另外，在即将发布的企业版 5.2 中，读放大问题也在代码层面进行了优化，内部测试显示 `safetensors` 的读放大基本能控制在 1 倍左右了。

社区版优化实践

JuiceFS 社区版在我们内部的应用更为广泛，涉及到数据采集、模型训练以及大数据的场景，因此我们对社区版做了较多优化。由于篇幅限制，本文将重点介绍万级客户端的稳定性优化以及 Writeback 写缓存相关优化工作，其他优化措施只做简要说明。

万级客户端下的稳定性

早期监控数据显示，当客户端数量增加至两三千时，TiKV 的负载迅速达到上限。如从下图所示，每个 TiKV 实例仅绑定一个磁盘，单实例读带宽已逼近 6 GB/s，基本耗尽 NVMe 盘吞吐能力。同时，监控中频繁出现 Region 分裂和 Prewrite Error，表明系统在高并发场景下存在严重的事务冲突问题。这些问题成为我们在大规模部署 JuiceFS 时面临的技术瓶颈，严重影响了系统的稳定性与横向扩展能力。

针对这个问题，我们采取了以下优化措施：

首先，发现 TiKV 的瓶颈主要源于大量高频的扫描任务。由于扫描操作对 TiKV 开销较大，我们延长了后台扫描任务的周期，降低了扫描频率。在 JuiceFS 1.3 之前的版本中，有些扫描任务甚至以分钟级频率运行，随着客户端数量增加，TiKV 压力迅速上升。

其次，**在读请求路径上，我们引入了元数据缓存，充分利用了内核的缓存机制。**Kernel 是我们的好伙伴，只要明确标示数据可安全缓存，Kernel 会自动启用相应策略，从而显著减轻对元数据引擎的访问压力。此外，在简单只读事务场景中，我们采用了 TiKV 的点读（Point Get）优化，绕过了不必要的分布式事务流程，显著减少了对 Placement Driver（PD）的访问，并将相关操作性能提升了约 50%。

对于写请求，我们解决了大量事务冲突。事务冲突在 TiKV 中会导致大量的写入错误。虽然 **JuiceFS 默认采用乐观锁来解决事务冲突**，但在冲突较多的情况下，乐观锁反而会起到副作用。我们还尝试将某些接口改为悲观锁，但由于改动较大，最终并未推进下去。

用增量更新吗？还是怎么搞呢？

值得一提的是，上述优化已全部合并进 JuiceFS 的社区版本，社区用户从 1.3 版本起即可直接受益。也算是我们为 JuiceFS 社区做了点微小贡献。

异构环境下的兼容性

由于机器来自不同厂商和采购渠道，硬件环境存在较大差异。例如，有些机器无数据盘，有些配有 NVMe 盘但数量有差异，有的未配置 RAID，甚至挂载点也不一致。这些差异为系统的统一管理与运维带来了显著挑战，异构硬件的兼容性也成为亟待解决的问题。

为此，我们也进行了额外的开发工作，通过 JuiceFS CSI Node 实现了对异构环境的良好适配。具体而言，CSI Node 作为每个节点的 DaemonSet 部署，能够感知所在节点的硬件信息，如磁盘类型、容量、内存等，并基于这些信息构建出 Mount Pod 的挂载参数，从而实现按节点特性灵活挂载。这一方案有效简化了异构环境下的存储管理工作，确保了系统在异构硬件平台上的一致性和性能表现。

S3 Gateway 支持多租户

该问题源于历史遗留工具对 S3 接口的强依赖。有些工具只能通过 S3 协议访问数据，而 JuiceFS 自带的 Gateway 默认一个实例仅能处理一个卷的请求。由于公司内部存在大量独立的卷（数量有数百个），为每个卷单独部署 Gateway 实例不仅资源开销大，也带来了巨大的运维负担。

为解决该问题，我们对 Gateway 进行了扩展，使单个实例支持多卷请求处理。用户只需通过统一的 Endpoint，便可像访问标准 S3 服务一样访问各自的卷，实现多租户环境下的资源复用，显著提升了系统的可扩展性与运维效率。同时，我们集成了公司内网的 IAM 鉴权系统，进一步强化了安全性和用户隔离能力。

冷热存间的数据沉降与预热

目前我们部署了一套大容量的 OSS 存储集群，并配有一套容量相对较小的热存储系统 GPFS（为公司早期采购，约 PB 级）。之前用户的训练任务，尤其是 Checkpoint 的读写，主要依赖于 GPFS。然而随着用户数据量快速增长，GPFS 容量持续趋紧，这使得存储系统的扩展面临挑战。因此，我们需要在冷热存储之间实现有效的数据沉降与预热，以确保系统的稳定性与性能。

为应对这一挑战，我们提出了一系列优化方案。首先，将 JuiceFS 的 cache-dir 指向 GPFS 挂载点，复用 JuiceFS 已有的缓存管理能力，实现热数据的自动迁移，从而更高效地完成冷热数据的分离与转移。

其次，我们进行了二次开发工作，保留了本地盘的支持，构建了**二级缓存机制**。数据优先从本地 NVMe 硬盘读取；若本地未命中，则回退访问 GPFS 或 OSS。这一策略显著提升了读性能，尤其在数据访问具有局部性的场景中，有效降低了访问延迟。

在读缓存之外，我们也支持了写缓存能力，特别是在 Checkpoint 数据写入过程中启用了 Writeback 模式，使写入过程具备更强的容错性与更高的吞吐表现，从而提升数据写入的可靠性与性能。

通过上述优化，我们实现了冷热存储间高效的[数据迁移](#)与访问加速，确保系统在扩展过程中的稳定性，满足不断增长的业务需求。

writeback 写缓存与跨机房数据访问优化

为了提升写入性能，我们强依赖于 JuiceFS 的 Writeback 机制。然而，默认的 Writeback 存在两个问题：

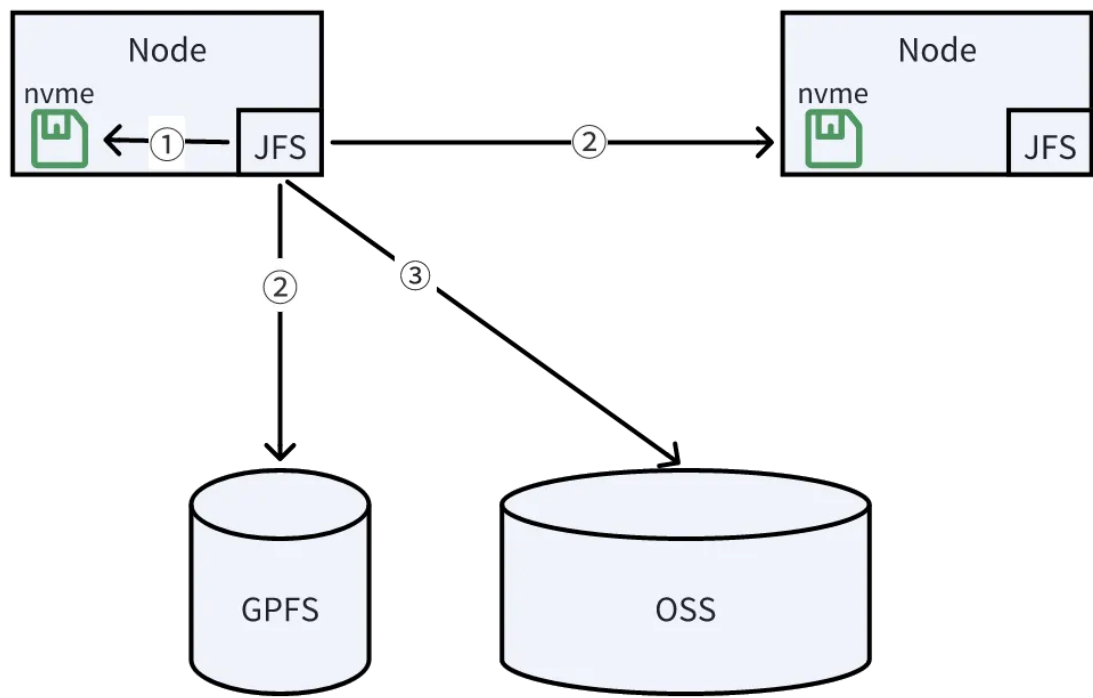
- 数据安全性**：Writeback 将数据写入本地磁盘后就给用户返回成功，此时若本地磁盘损坏或进程异常退出，存在数据丢失的风险。
- 数据可见性**：在数据仍滞留于本地缓存、尚未上传至 OSS 时，其他节点无法访问这部分数据。在堆积量较大情况下，可见性延迟甚至可能达到小时级别。

针对这两个问题，我们将 Writeback 缓存目录配置在具备容灾能力的 GPFS 上，使缓存数据直接落盘至稳定的[共享存储](#)。这样不仅保障了数据安全性，也实现了跨节点的数据实时可见。

此外，该设计还带来了额外收益：通过在 GPFS 上叠加 FUSE 层，Writeback 机制屏蔽了底层存储的异构差异，形成类似阿里云 CPFS 的架构模式，使系统能够无缝接入除 GPFS 以外的其他[高性能存储](#)，实现统一透明的存储访问能力。

在跨机房部署场景下，该方案仍面临 Writeback 的两个核心问题：由于 GPFS 没有跨机房部署，当一个机房写入数据后，其他机房在数据同步到 OSS 之前将无法访问，导致数据可见性问题依然存在。

为解决多机房间的数据访问延迟，我们新增了 P2P 读取功能，实现跨机房数据共享。最终的读请求流程如下：



图片

数据读取流程

1. 优先读取本地磁盘：若数据存在于本地，直接返回。
2. 并行查询 GPFS 和 P2P 节点：同时向 GPFS 与远端 Peer 节点发起请求，使用最先返回的数据源。
3. 回退至 OSS：若上述两者均未命中，则回退至 OSS 获取数据。

通过该机制，在保障性能的同时，提升了跨机房场景下的数据可用性与访问效率。

我个人从 2021 年 JuiceFS 推出开源版本起便开始关注并参与该项目。自最初在推理场景中引入 JuiceFS 起，该系统已逐步发展为我们内部统一的文件系统接入层，覆盖数据生成、训练、推理等大模型研发全流程的核心环节。其基于对象存储的架构显著降低了我们的数据存储成本，良好的兼容性也使其能够顺利集成进多云和异构环境下的各类系统。这些特性恰好契合了我们对灵活性、稳定性与成本效率的核心诉求。希望本文的实践经验能为同业在大模型基础设施建设中提供一些参考，欢迎交流探讨。